

# Deixei o Kiro Crew cuidar do pior "servidor" do mundo!

Agentes autônomos cuidando do que eu não queria mais cuidar!

Felipe KiKo

# Quem é o KiKo?

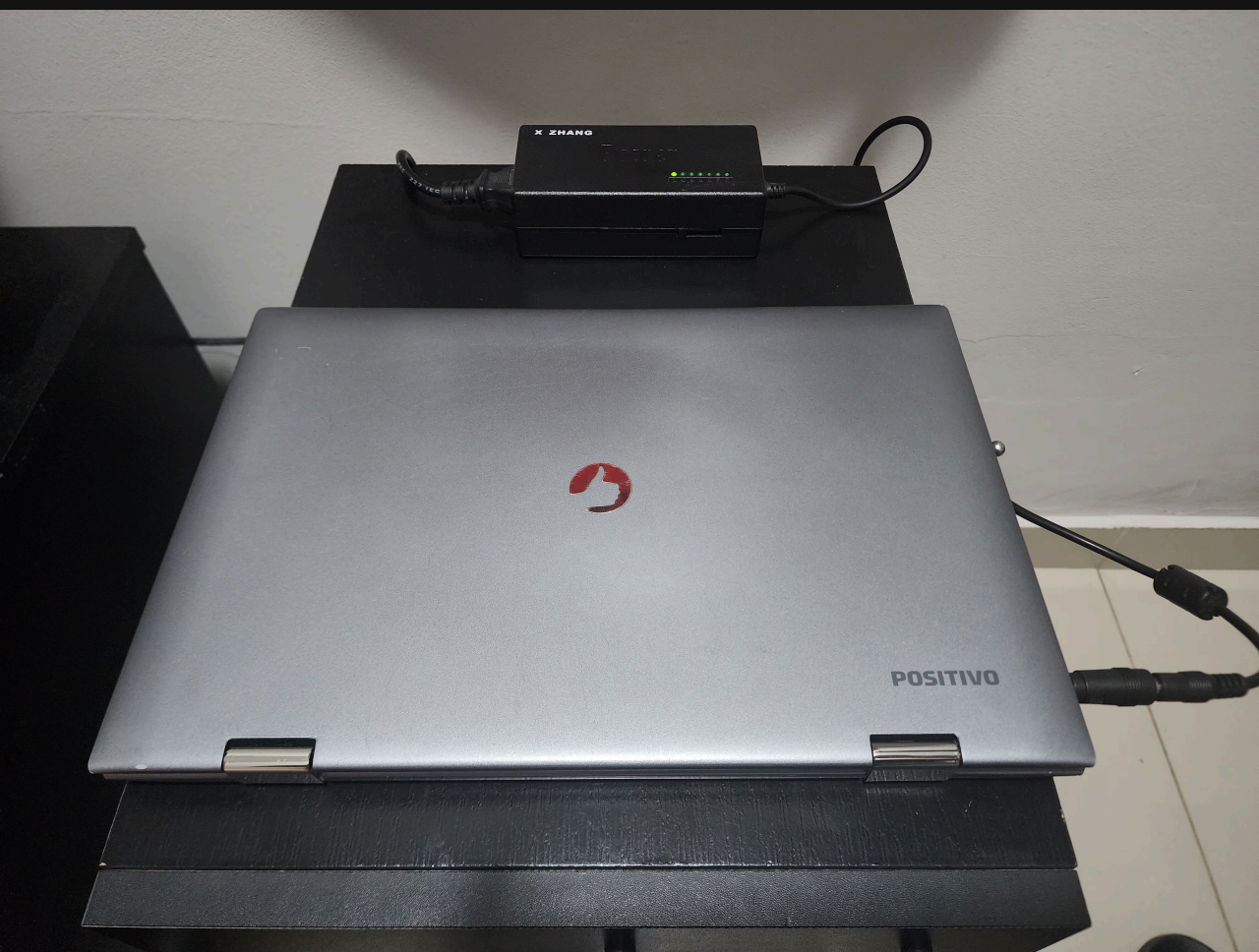
- Cloud Architect, Developer e FinOps @ **Avantpro**
- +20 anos em Tech e colecionador de problemas!
- **AWS Community Builder** (Dev Tools)
- Leader do **AWS User Group Campinas**

Movido a curiosidade, desafios, Pokémon e café

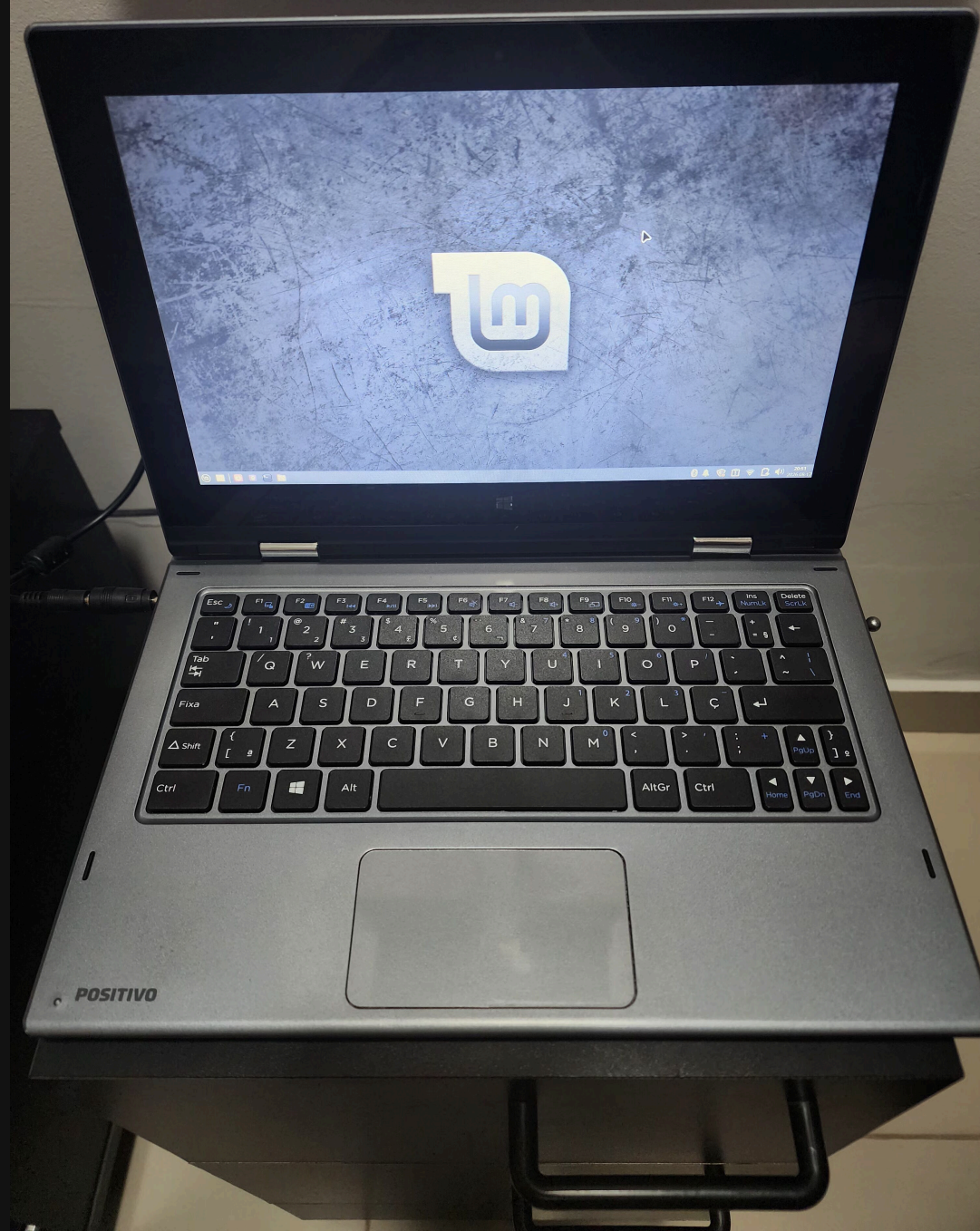


# O pior "servidor" do mundo!

*Sim...tadinho dele...*



\* O "SERVIDOR"



# Positivo DUO ZR3630

## AS SPECS QUE CHORAM!

| COMPONENTE | SPEC                 |
|------------|----------------------|
| CPU        | Intel Celeron        |
| RAM        | 4 GB                 |
| Disco      | 32 GB flash          |
| OS         | Linux Mint 22 (XFCE) |

## PARA QUE ELE "SERVE"

- Experimentos **beeem** limitados
- Emprestar pro afilhado em workshops
- Me fazer passar **raiva**

# Mas nele roda algo importante...

## SISTEMA DE CONTROLE DE GASTOS PESSOAIS

- **Backend:** Python
- **Banco:** MySQL
- **Infra:** Docker containers
- **Acesso:** Tailscale (SSH remoto)

# O problema real

## ❶ Trava do nada

Com 4 GB de RAM...qualquer coisa fora do padrão e morre tudo

## ❷ SSH manual pra rebotar

Cada incidente = eu tenho que entrar lá e ver qual foi a da vez...

## ❸ Queries manuais pra relatórios

Todo dia, entrar via SSH e rodar SELECT para ver meus gastos

## ❹ Zero automação

Tudo na mão...até agora

**O que é o Kiro Crew?**

# Kiro Crew em uma frase

*"Um workspace de desenvolvimento persistente e open source que lembra seu contexto, aprende como você trabalha e coordena agentes autônomos, tudo dentro da sua máquina!"*



Memória entre  
sessões



Cron, Webhooks  
Heartbeats



Open Source  
Apache 2.0

# O que faz ele diferente?

## MEMÓRIA E APRENDIZADO

- **Skills:** Workflows viram habilidades permanentes
- **Lessons:** Correções viram aprendizado
- **Knowledge Graph:** Decisões e contexto indexados
- Tudo em Markdown, editável, inspecionável

## TRABALHA ENQUANTO VOCÊ DORME

- **Cron schedules** com timezone
- **Webhooks** autenticados
- **Heartbeat monitors** em PRs e pipelines
- **7 camadas de segurança** (sandbox, auditoria, redação de credenciais)

# O Setup

*Kiro Crew + Positivo DUO = coragem*

# Arquitetura da solução

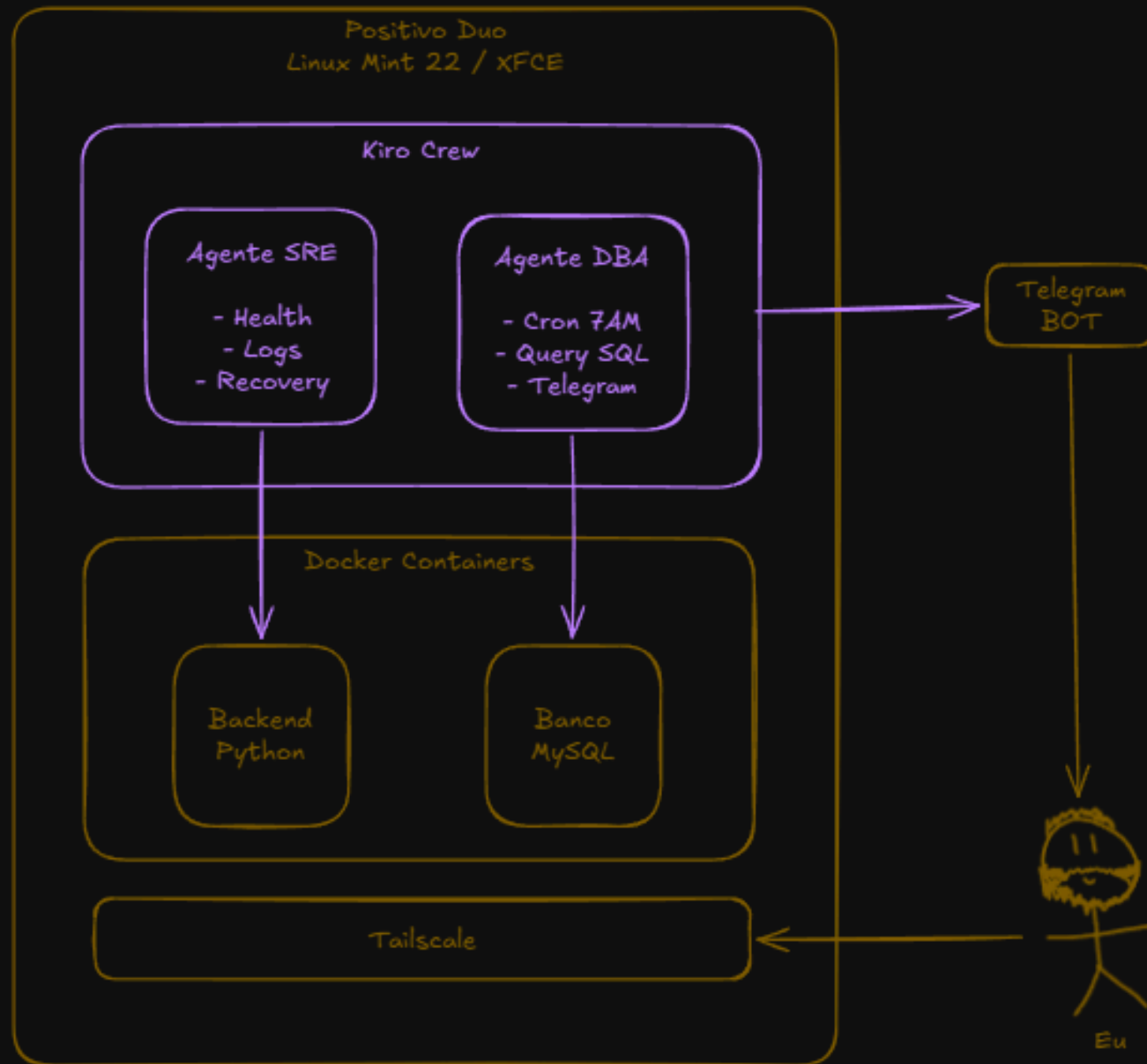
## NO HOST

- **Kiro Crew** rodando direto no SO
- Acesso ao **docker** CLI
- Monitora recursos do host
- Se Docker morrer → Crew reinicia

## NOS CONTAINERS

- **Backend** (Python) — app de gastos
- **MySQL** — dados financeiros
- Health checks expostos
- Logs acessíveis ao Crew

**Por que fora do Docker?** Se ficasse dentro do container, morreria junto com ele. Precisa estar no host para poder salvar o Docker.



# Os dois agentes criados



## AGENTE SRE

- Verificar se containers estão de pé
- Monitorar health checks
- Analisar logs em caso de meleca
- Tentar recuperar o serviço **sozinho**
- Me avisar só se precisar de ajuda



## AGENTE DBA

- Todo dia às 7h da manhã
- Consultar o banco MySQL
- Rodar a query de gastos
- Processar e formatar
- Mandar no Telegram com resumo

# SRE: Hora de maltratar o "server"

*Simulando incidentes que "raramente" aconteciam todos os dias*

# Incidente 1: Matei o backend

```
docker stop backend
```



Detectou



Causa raiz



Recuperou

Container parado detectado imediatamente. `docker start backend` executado com sucesso.

# Incidente 2: Matei o banco

```
docker stop mysql
```

Backend de pé...mas quebrado!



Detectou via logs  
da aplicação



Identificou que o MySQL  
era a causa



Recuperou banco  
+ reiniciou backend

# Incidente 3: Esgotar a memória

## O TESTE MAIS INTERESSANTE

Com 4 GB de RAM...foi fácil simular 💀

### SURPRESA 1

O Kiro Crew morreu junto!

(era esperado com 4 GB)

### SURPRESA 2

Quando **voltou à vida**, ele:

- Entrou em alerta **antes** de agir
- Fez diagnóstico completo
- **Corrigiu o esgotamento de memória!**

# Como ele "corrigiu" memória?

Não fez download de RAM magicamente... 🤔

## 1 Memory limits nos containers

Configurou limites para que os containers não consumissem toda a RAM do host

## 2 Travas de proteção nele mesmo

O Crew colocou limites no próprio consumo para não morrer de novo

## 3 Documentou a ação

Registrou o que aconteceu e o que fez...lesson learned persistida!

Ele não só resolveu...ele **aprendeu** para não acontecer de novo.

# DBA: A mensagem das 7h01

*Me lembrando diariamente das minhas más decisões financeiras*

# O que eu fazia antes

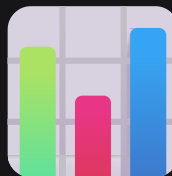
```
ssh positivo  
docker exec -it mysql mysql -u root -p  
  
SELECT date, description, amount  
FROM expenses  
WHERE date = CURDATE() - INTERVAL 1 DAY;
```

Resultado cru → interpretação **mental** → ansiedade 

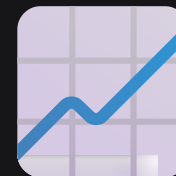
# O que o Crew me entrega agora



Gastos detalhados  
do dia anterior



Acumulado  
do mês



Projeção  
de custos!

Tudo que eu fazia "mentalmente" com o resultado da query, ele me trouxe ali, na palma da mão, sem eu precisar fazer nada!

# Demo time!

*Vamos ver o Crew em ação...*

# O que vamos ver ao vivo

## 1 O setup no Positivo

Kiro Crew rodando, agentes ativos, dashboard

## 2 Os agentes em operação

SRE monitorando, DBA com schedule configurado

## 3 Simulação de incidente

Matar um container e ver a mágica acontecer

## 4 A mensagem do Telegram

O relatório financeiro que chega todo dia de manhã

# Considerações finais

*O insight que mudou minha cabeça*

# A pergunta que eu me fazia

*"Beleza, mas não daria para fazer isso com Kiro CLI? Ou com uma automação do Kiro Web?"*

A resposta: **sim**, individualmente quase tudo dá para reproduzir de outra forma.

Mas o que mudou foi parar de pensar no Crew como "um agente melhor"...

...e começar a pensar nele como um lugar onde agentes ficam **responsáveis** por coisas que estão no meu domínio

| ABORDAGEM | MENTALIDADE                         |
|-----------|-------------------------------------|
| CLI       | "Investigue por que minha API caiu" |
| Crew      | "Essa API agora é problema seu"     |

Cada um assume suas responsabilidades. CLI = você pede. Crew = ele cuida.

# O momento "WOW"

Depois de alguns dias...eu esqueci que ele existia.

**7:01**

Mensagem chegava  
todo dia



Aplicação  
funcionando



Tudo num Positivo  
na minha mesa

**Virou natural. E isso é o ponto.**

# Desafio pra vocês!

*Qual aquele serviço, script, rotina ou servidor que você cuida manualmente hoje...e que poderia virar responsabilidade de um agente?*

- Aquele cron que quebra e ninguém vê
- Aquela VM que precisa de reboot toda semana
- Aquele relatório que alguém roda na mão todo dia
- Aquele health check que só funciona quando você lembra

# Obrigado!



## LINKS E REFERÊNCIAS

- Site: [kiro.dev/crew](https://kiro.dev/crew)
- GitHub: [github.com/kirodotdev/KiroCrew](https://github.com/kirodotdev/KiroCrew)

Dúvidas: Me chama!